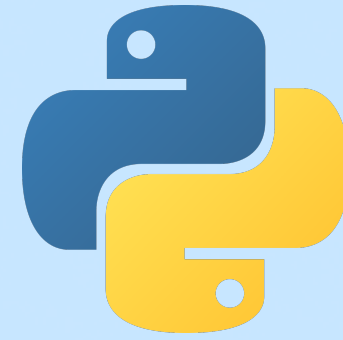


Introduction and basics

Command line, Python, Jupyter

Amine Ferdjaoui
amine.ferdjaoui@sogeti.com

Contenu du cours

1. Manipulation de la ligne de commande via terminal.

- a. Chemins
- b. Commandes Bash
- c. Editeur de text VIM

2. Python

- 1. Pip et venv

[New] 2. Uv

[New] 3. Outils de Coding avec IA dans VS Code

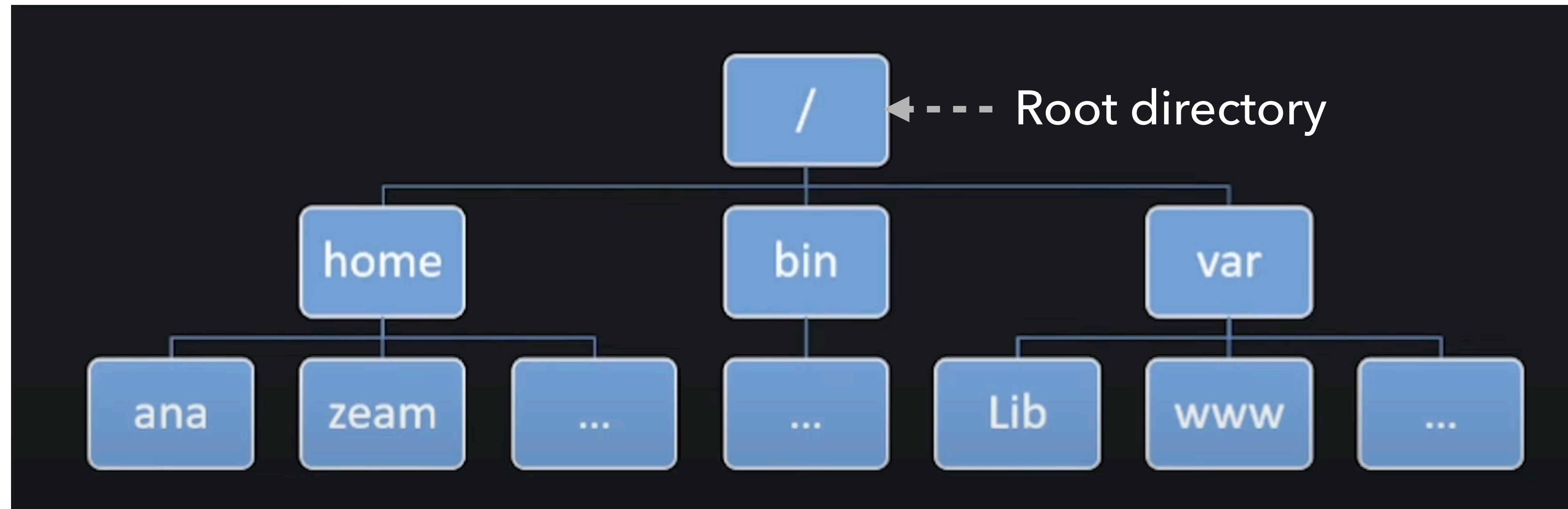
3. Jupyter et *voilà*

A vertical blue gradient bar on the left side of the slide, transitioning from a darker blue at the top to a lighter blue at the bottom.

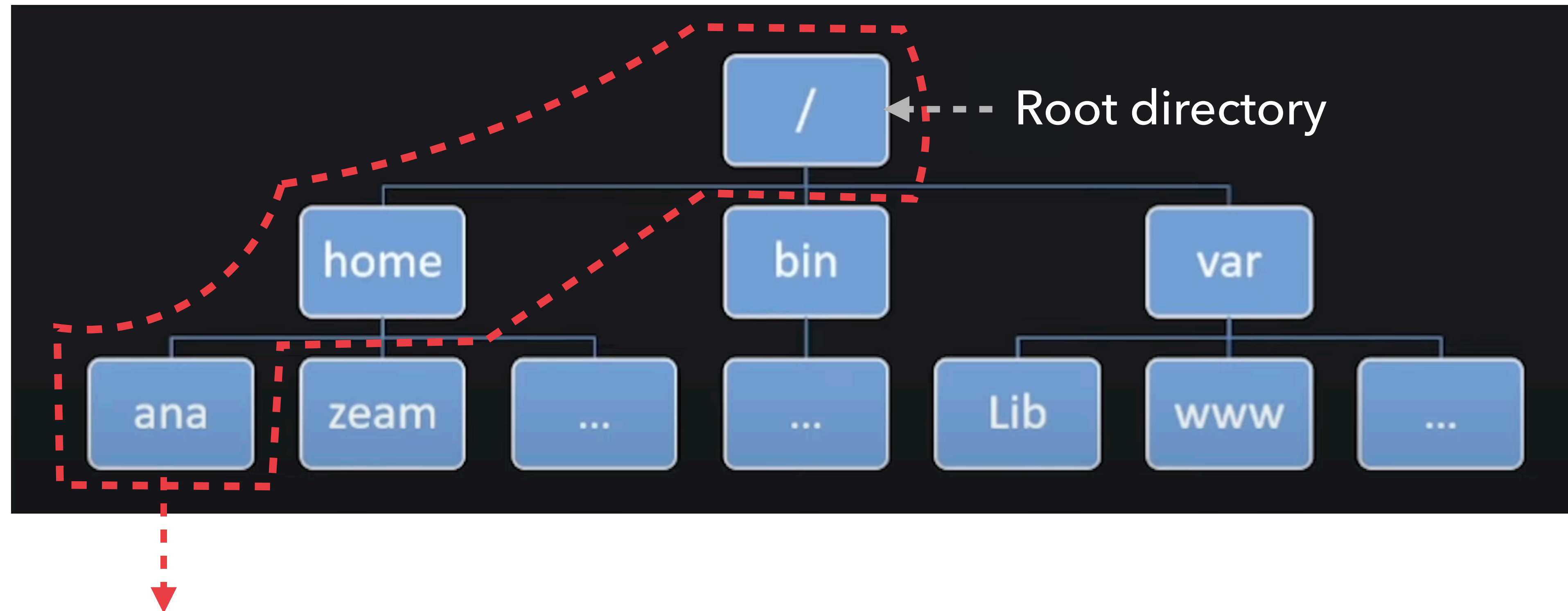
Linux

Command line

Linux file system

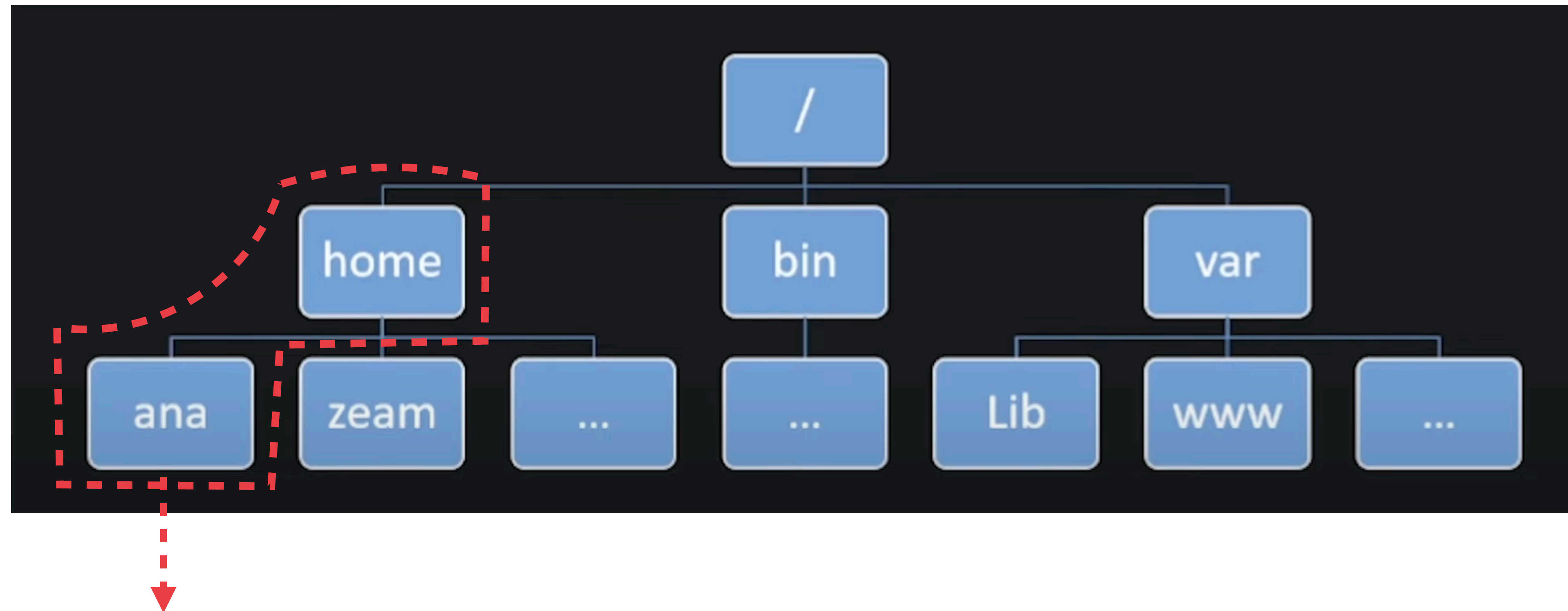


Linux file system



Absolute path **/home/ana** (Full path from root)

Linux file system



Relative path (Depends on your current directory)

Command line basics

```
$ pwd    #Print working directory
```

```
$ cd <dir> #Change directory
```

```
$ cd ..  #Change to parent directory
```

```
$ ls -A  #List
```

```
$ mkdir <dir(s)> #Create one or multiple directories
```

```
$ touch <file(s)> #Create one or multiple files
```

Command line basics

\$ pwd #Print working directory

\$ cd <dir> #Change directory

\$ cd .. #Change to parent directory

\$ ls -A #List

\$ mkdir <dir(s)> #Create one or multiple directories

\$ touch <file(s)> #Create one or multiple files

\$ cp <src> <dest> #Copy file

\$ cp -r <src> <dest> #Copy directory

\$ mv <src> <dest> #Move/rename file

\$ rm <file> #Remove file

\$ rm *.txt #Remove all text files

\$ rm -r <dir> #Remove directory

Command line basics

```
$ history #history > hist.txt
```

```
$ clear
```

```
$ echo
```

```
$ chmod #Change file permissions
```

```
$ grep #history | grep cd
```

```
$ diff #Compare two text files line by line
```

Command line basics

```
$ man <command> #Manual page
$ locate <file> #Locate a file in the system
$ cat <file> #Display a file in the terminal
$ open . #(or start .) display the current folder
$ head #Show the first lines of a file
$ tail #Show the last lines of a file
$ exit #Exit the current shell/terminal session
$ ps # (Process Status) Show the list of running processes
$ kill #Terminate a process by its Process ID (PID)
```

```
$ history #history > hist.txt
$ clear
$ echo
$ chmod #Change file permissions
$ grep #history | grep cd
$ diff #Compare two text files line by line
```

1. Create your workspace

- Go to your **home directory** (`pwd` to check).
- Create a folder named `M1_project` (`mkdir`).
- Inside it, create 3 subfolders:
 - `data/` → for datasets
 - `src/` → for Python scripts
 - `notebooks/` → for Jupyter notebooks

2. Add some files

- Inside `src/` create two files: `main.py` and `utils.py` (`touch`).
- In `data/` create two files: `train.csv` and `test.csv`.
- Verify that everything is created (`ls -A`).

3. Organize the files

- Move `main.py` into the root folder `M1_project/` (`mv`).
- Copy `utils.py` into `notebooks/` (`cp`).
- Rename `train.csv` into `training.csv`.

4. Clean up mistakes

- You don't need `test.csv`. Delete it (`rm`).
- Also, remove the `utils.py` inside `notebooks/` using `rm`.

5. Explore history and filters

- Show the list of your previous commands (`history`).
- Save them into a file called `history.txt`.
- From this file, extract only the commands where you used `cd` (`grep`).

Use VIM

VI (**V**isual editor) and VIM (**V**i **I**Mproved) are default text editor on almost all Unix/Linux systems.

```
$ vim main.py
```

The modes are shown at the bottom left of the editor

- The **command mode** is the default mode.
- To go back to command mode type **esc** key.
- To start modifying (**Insert mode**) type **i** key on keyboard.
- To **exit**, go back to command mode and
 - no modifications, type **:q**
 - modified file
 - save modifications **:wq**
 - exit without save **:q!**

Python

Virtual environment

When you start a project locally you should always work with **virtual environments (venv)** or conda.

To initialize an empty virtual environment :

```
$ python -m venv venv
```

To start it :

Unix (mac or linux)

```
$ source venv/bin/activate
```

On windows

```
$ .\venv\Scripts\activate
```

To exit :

```
(venv) $ deactivate
```

How to use python?

- Command line REPL (Read-Eval-Print Loop)

\$ python

```
Python 3.12.13 (main, Mar 25 2026, 03:16:06) [Clang 22.1.1 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

\$ ipython

```
Python 3.12.13 (main, Mar 25 2026, 03:16:06) [Clang 22.1.1 ]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.17.1 -- An enhanced Interactive Python. Type '?' for help.
Tip: You can change the editing mode of IPython to behave more like vi, or emacs.

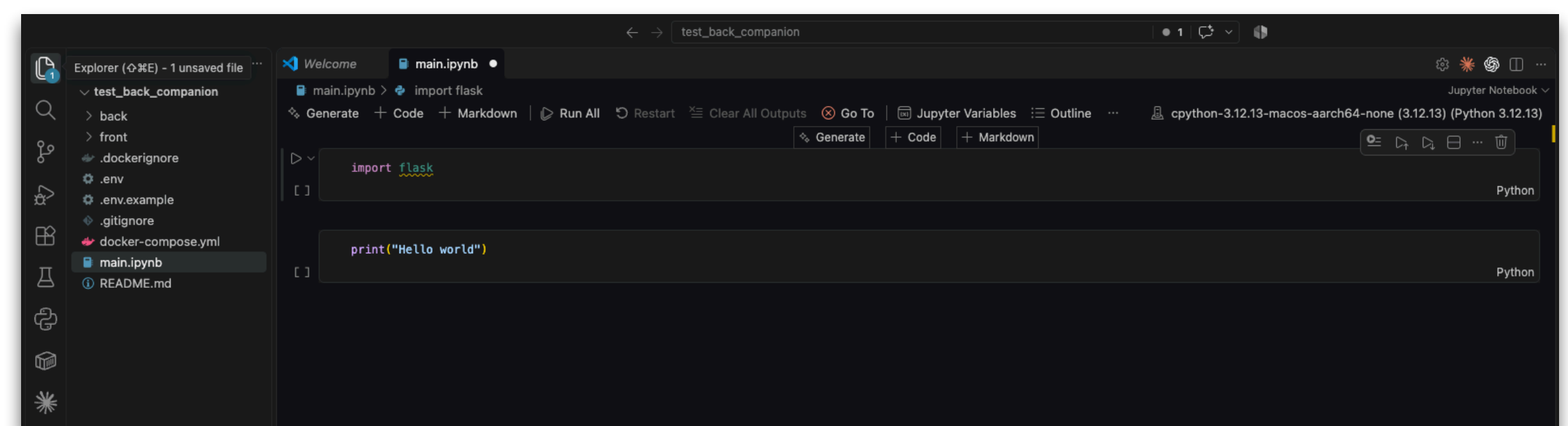
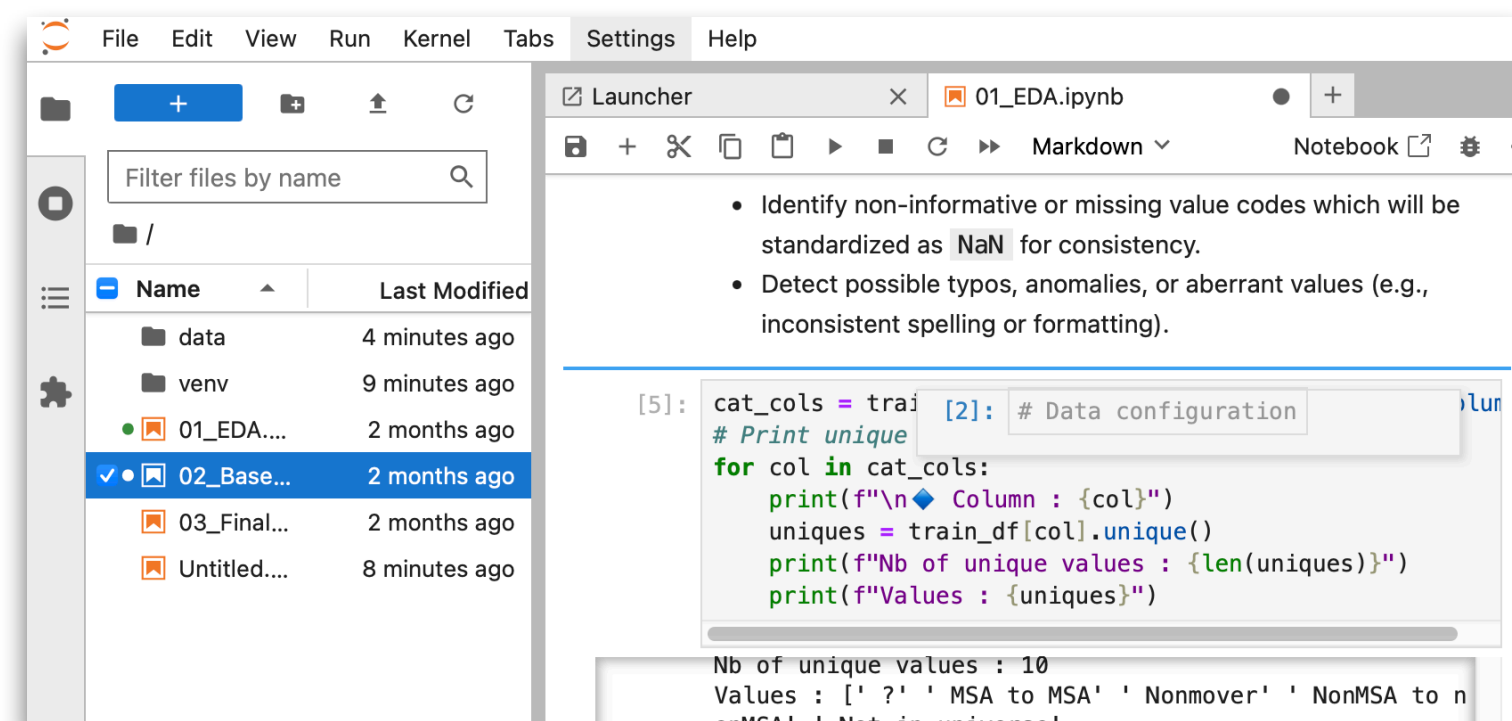
In [1]: █
```

- Python scripts (.py files)

\$ python main.py

- Jupyter Notebook / .ipynb files

IDEs, like PyCharm, VS Code.



PIP : Package Installer for Python

Install a library

```
$ pip install <library_name>
```

```
$ pip install pandas
```

```
$ pip install pandas==2.2.2
```

Install a library list from file

```
$ pip install -r requirements.txt
```

requirements.txt

```
1 matplotlib==3.10.6
2 numpy==1.26.4
3 pandas==2.2.2
4 Flask==3.1.2
```

Show installed packages

```
$ pip freeze
```

```
$ pip list
```

Remove a library

```
$ pip uninstall <library_name>
```

```
$ pip uninstall pandas
```

Context : You need to write a **Python script** for the project. But you must do it properly:

- edit it using Vim,
- run it in a **virtual environment**,
- manage dependencies with `pip`,
- and control file permissions with `chmod`.

1. Write your first script with Vim

- Go inside the `M1_project/` folder created in TP1.
- Create a new file `hello.py`. 
- Open it with Vim and write a simple program

2. Make the script executable

- Check permissions using `ls -l`
- Add executable permission using `chmod +x`
- Remove the executable permission and try to run again.
- Re-add the executable permission.

3. Create a virtual environment and install libraries

- Initialize a venv inside the project.
- Inside the venv, install pandas.
- Freeze the environment into a requirements file.
- Deactivate the venv and Create a new venv and reinstall from requirements.

```
import sys
import pandas as pd

def greet(name):
    print(f"Hello {name}, welcome to M1 project!")

def show_grades():
    df = pd.read_csv("data/students.csv")
    print("Students and grades:")
    print(df)

if __name__ == "__main__":
    if len(sys.argv) > 1:
        if sys.argv[1] == "grades":
            show_grades()
        else:
            greet(sys.argv[1])
    else:
        print("Hello World")
```

TP2 : First Python Script with Virtual Environment

Context : You need to write a **Python script** for the project. But you must do it properly:

- edit it using Vim,
- run it in a **virtual environment**,
- manage dependencies with `pip`,
- and control file permissions with `chmod`.

1. Write your first script with Vim

- Go inside the `M1_project/` folder created in TP1.
- Create a new file `hello.py`.
- Open it with Vim and write a simple program

2. Make the script executable

- Check permissions using `ls -l`
- Add executable permission using `chmod +x`
- Remove the executable permission and try to run again.
- Re-add the executable permission.

3. Create a virtual environment and install libraries

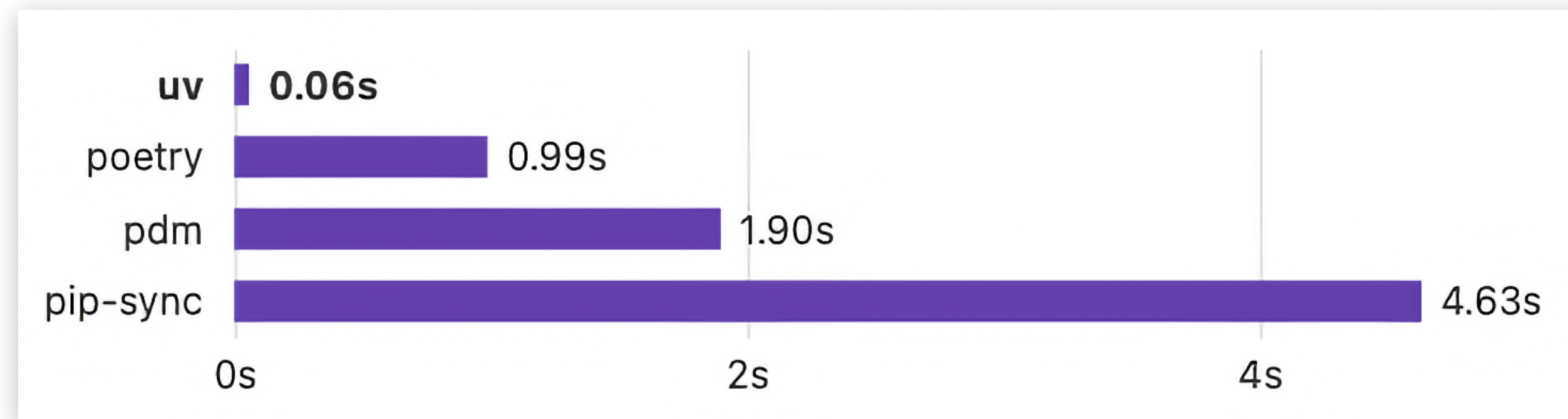
- Initialize a venv inside the project.
- Inside the venv, install pandas.
- Freeze the environment into a requirements file.
- Deactivate the venv and Create a new venv and reinstall from requirements.

```
ls -l hello.py
chmod +x hello.py
./hello.py Amine
chmod -x hello.py
```

```
pip install numpy pandas
pip freeze > requirements.txt
python -m venv venv2
source venv2/bin/activate
pip install -r requirements.txt
```

Python - uv

- An extremely **fast** Python package and project manager.
- It automates and makes the environment, dependencies, execution cycle **easier** and more **reliable**.



Classic workflow

- ◆ create a venv
- ◆ activate it
- ◆ install with pip
- ◆ export requirements.txt
- ◆ hope to reproduce the same environment

Uv

- ◆ creates/manages .venv
- ◆ adds dependencies
- ◆ runs code in the right environment
- ◆ locks exact versions
- ◆ speeds up installs with cache

Pyproject.toml: declared dependencies

uv.lock exact resolved versions

.venv/ local project environment

Python - uv essential commands

Set python version:

```
$ uv python install 3.12  
$ uv python pin 3.12 #tells the project which Python version to use
```

Create / initialize a project:

```
$ uv init my_project #for an existing project use only uv init
```

To add or remove a dependency only use:

```
$ uv add flask  
$ uv remove flask
```

→ Dependencies are recorded in
`pyproject.toml` and resolved in
`uv.lock`.

To add or remove a dependency only use:

```
$ uv run main.py
```


AI coding tools

1) Classic prompting workflow

AI interface → copy/paste → local execution



Often missing: environment setup • dependencies • tests • README • Git hygiene

Good for exploring an idea, but the developer must rebuild the project context manually.

2) In the IDE / VS Code

Agent works inside the project: files, terminal, tests, Git

VS Code / local project

```
src/app.py
pyproject.toml
uv.lock
tests/test_app.py
```

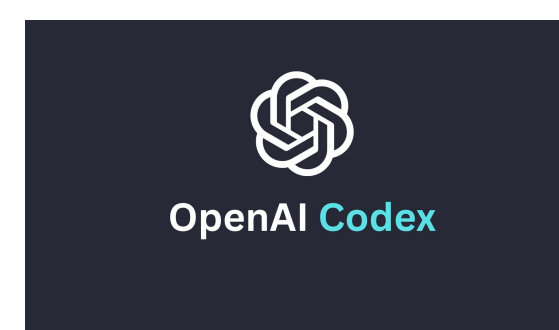
Agent actions

- edits files
- runs commands
- explains errors
- proposes fixes

Stronger workflow — but still requires supervision:

check requirements, review diffs, run tests, control versions.

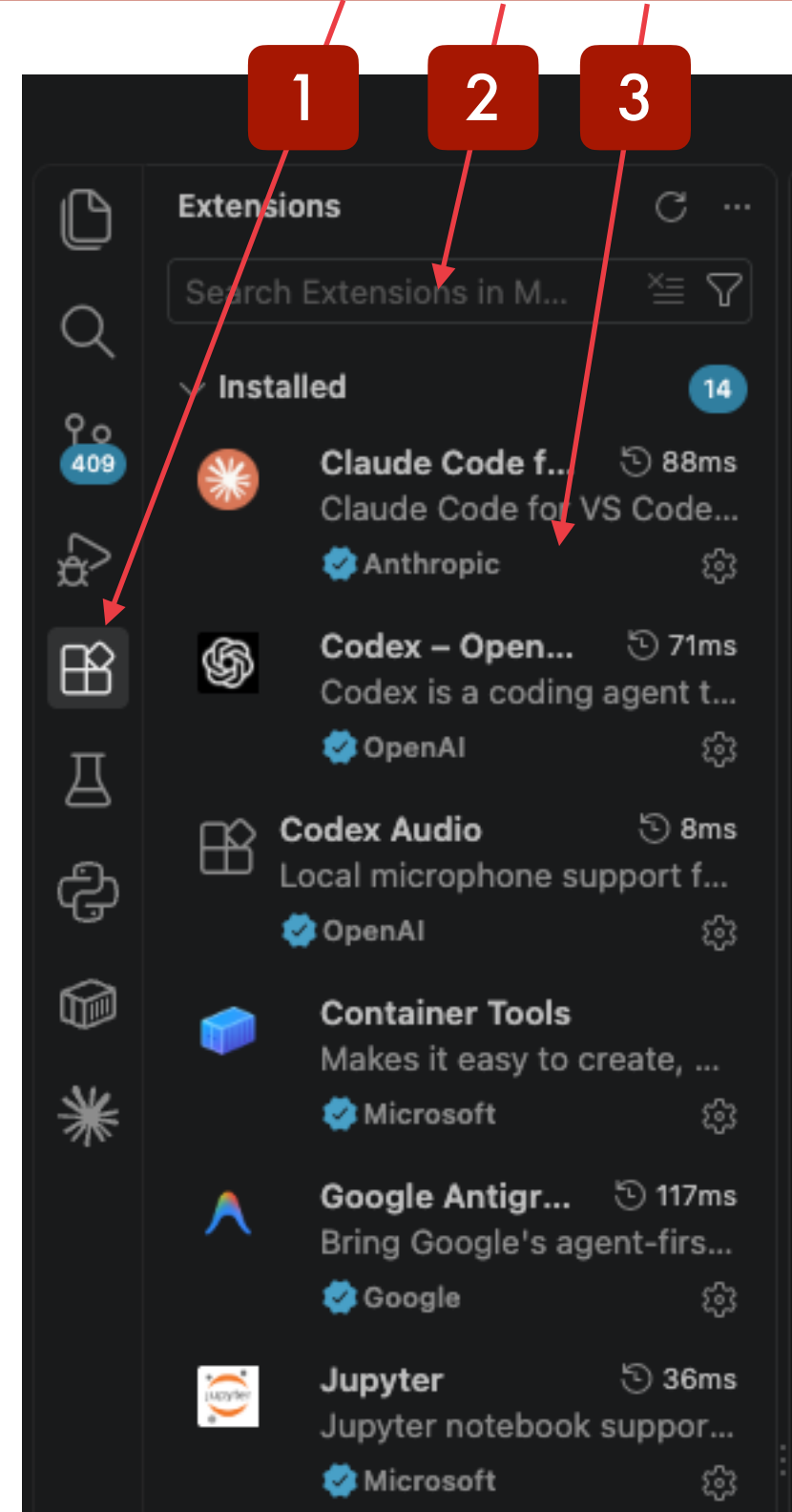
Representative coding agents / providers



AI can generate and modify code, but you remain responsible for the project: environment, dependencies, reproducibility and version control.

AI coding tools

Add extension here



Open the AI panel

